

Fundamentos de ML

Lecture06 – Regularization & KNN



Contenido

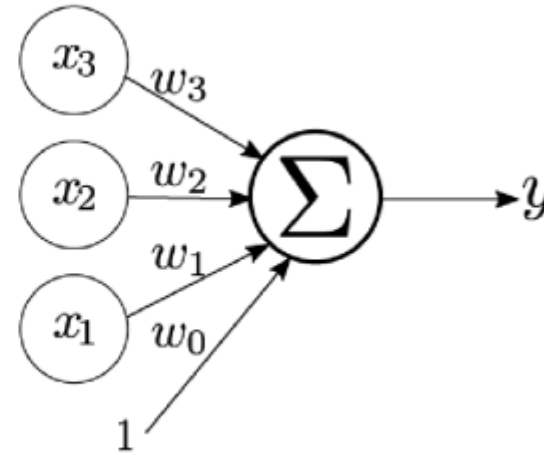
1. Repaso
2. Overfitting y Underfitting
3. Que es la regularizacion?
4. KNN
4. Pipelines
5. Practica de clase



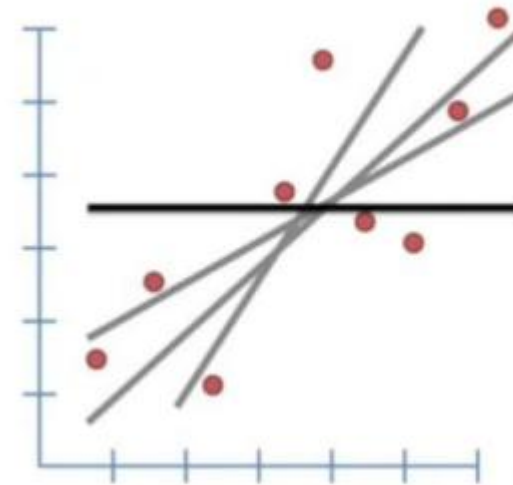
Repaso

$$\hat{y} = \hat{f}(\mathbf{x}) = w_0 + \sum_{j=1}^d w_j \times x_j$$

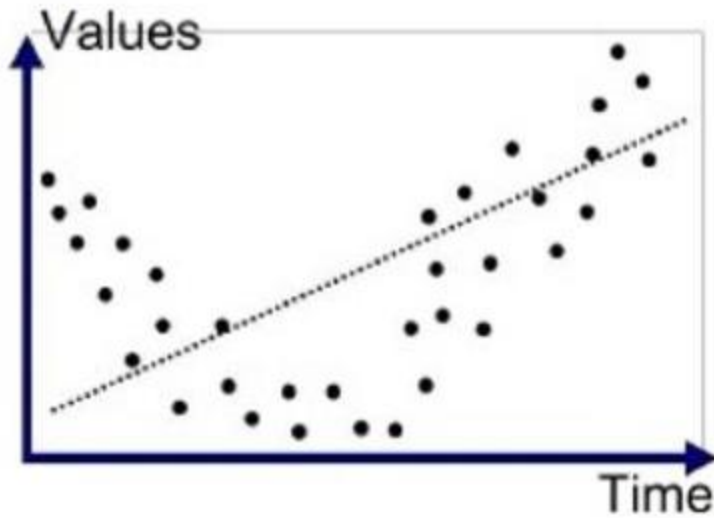
$$\hat{y} = \mathbf{w}^T \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}$$



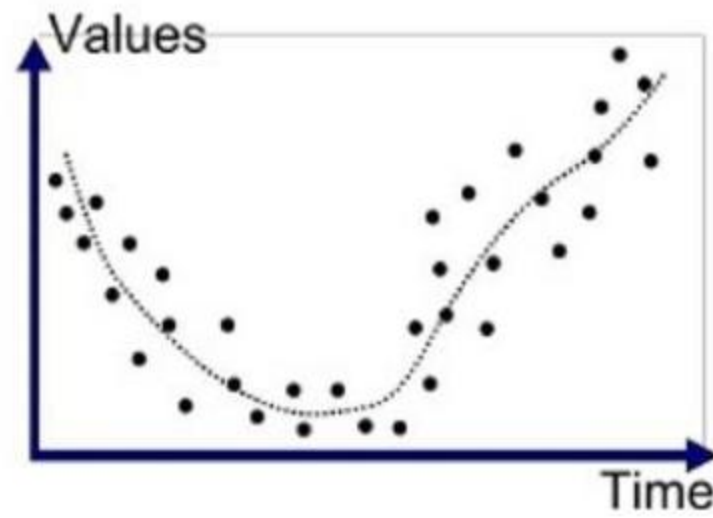
$$Cost_{\mathbf{w}} = \frac{1}{m} \sum_{i=1}^n \left(y_i - \mathbf{w}^T \begin{bmatrix} \mathbf{x}_i \\ 1 \end{bmatrix} \right)^2$$



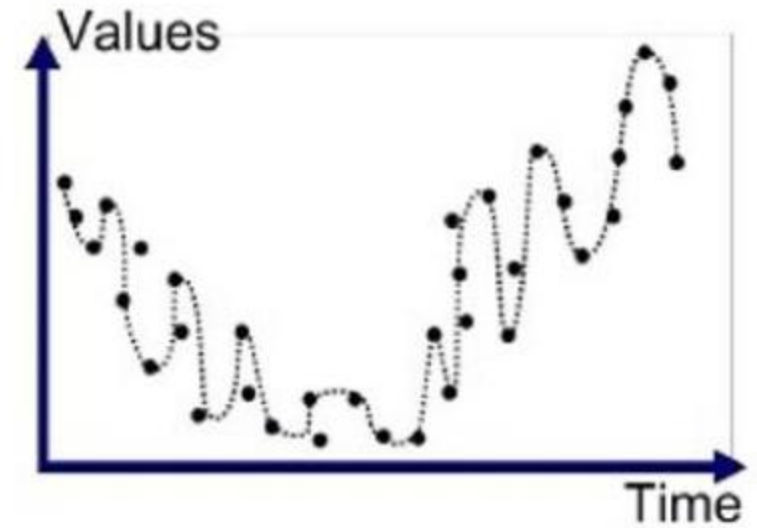
Overfitting and Underfitting



Underfitted



Good Fit/Robust

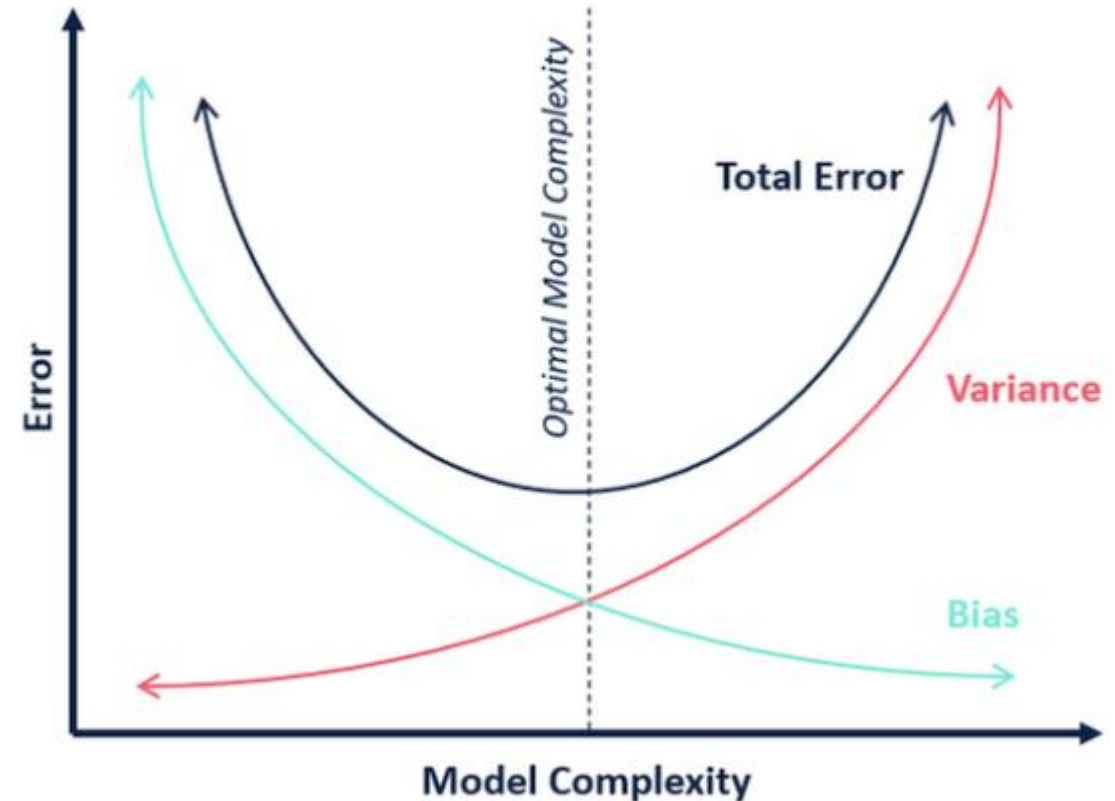


Overfitted

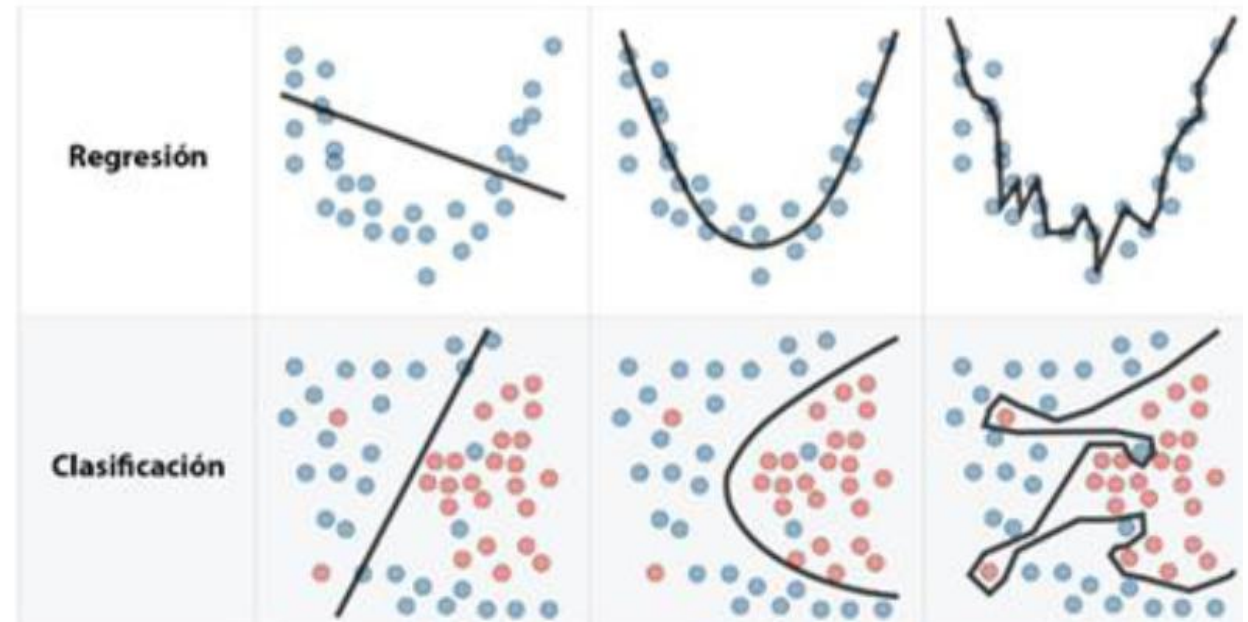
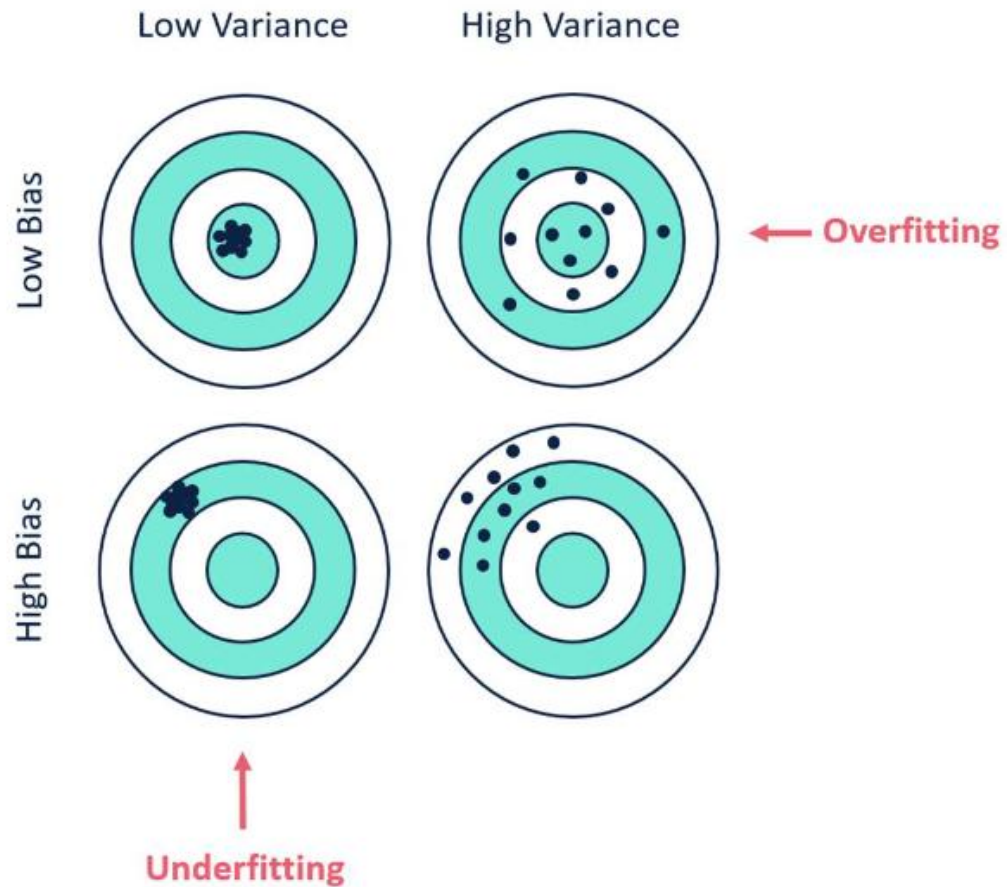


Overfitting and Underfitting

Por lo general, asociamos el sesgo alto con modelos muy simples que subajustan (underfit) los datos y asociamos la varianza alta con modelos muy complejos que sobreajustan (overfit) los datos



Overfitting and Underfitting



Regularización

- Técnica para evitar evitar el overfitting.
- Su función principal es "frenar" al modelo para que no se vuelva demasiado complejo y no memorice el ruido de los datos de entrenamiento.
- Es una penalización que se le aplica al mdoelo por tener coeficientes (pesos) demasiado grandes o por usar demasiadas variables.

Qué sucede si los coeficientes son muy grandes?



Regularización

Qué busca la regularización?

- Opción A (Sin regularización):

$$y=0.1x_1+10,000x_2$$

- Opción B (Con Ridge/Lasso):

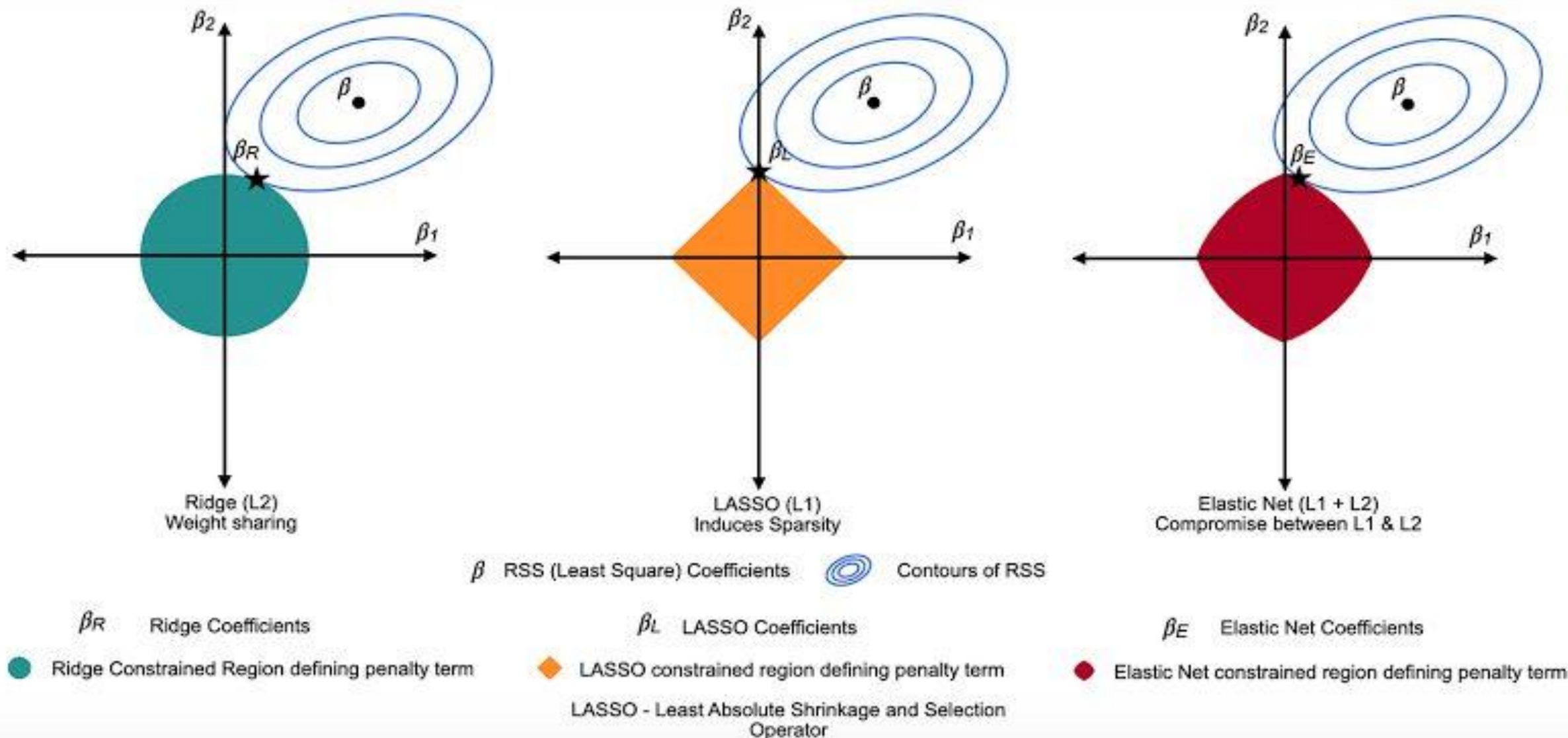
$$y=0.5x_1+0.8x_2$$

La Opción B es preferible porque es más simple y robusta. Si x_2 varía un poco por error humano, la opción A arruinará la predicción, mientras que la B apenas se inmutará.



Regularización

Tipos de regularización

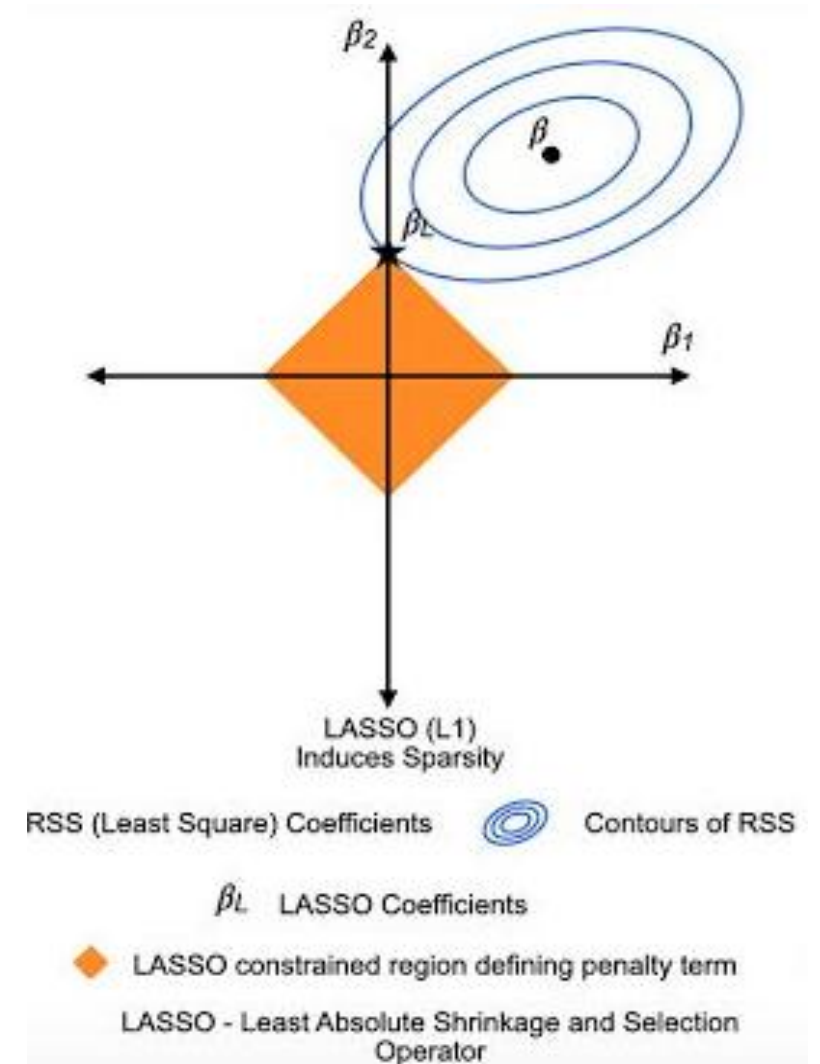


Lasso

La función de costo añade una penalización basada en el valor absoluto de los coeficientes

$$J(\beta) = \text{MSE} + \alpha \sum_{j=1}^n |\beta_j|$$

Lasso puede forzar a que algunos coeficientes sean exactamente cero. Esto significa que Lasso decide que algunas variables no son importantes y las elimina por completo del modelo.



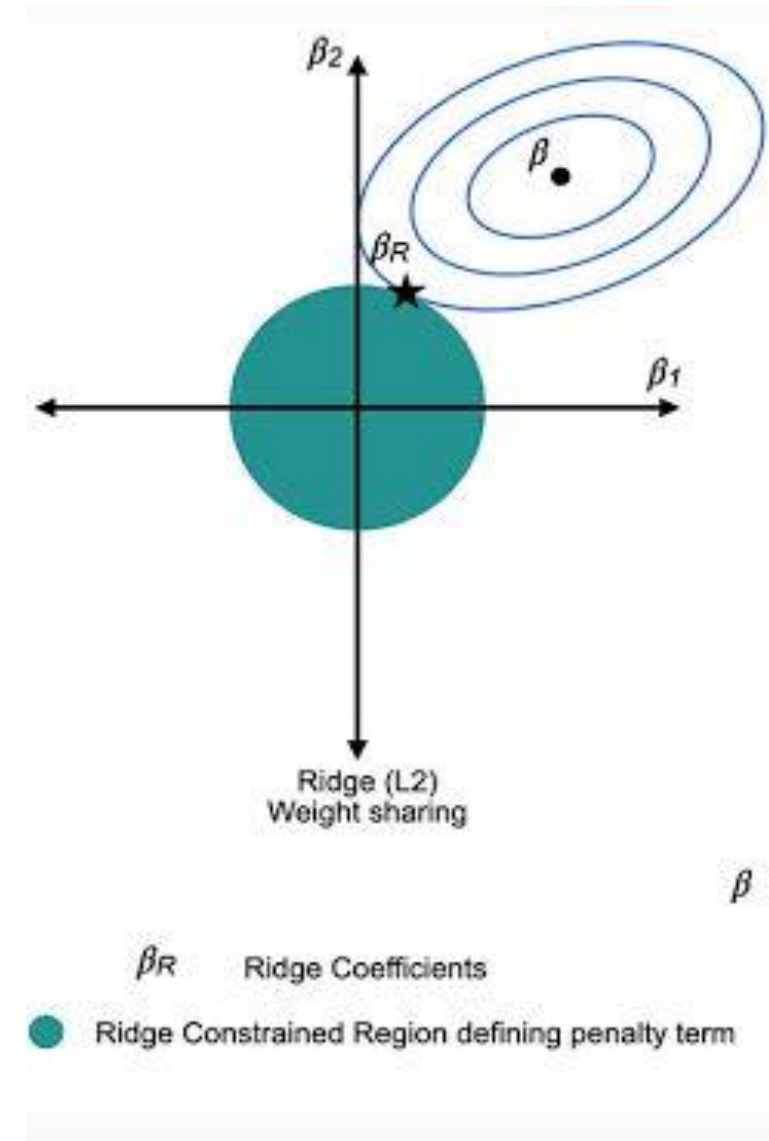
Regularización

Ridge

La función de costo añade una penalización basada en el cuadrado de los coeficientes.

$$J(\beta) = \text{MSE} + \alpha \sum_{j=1}^n \beta_j^2$$

A diferencia de Lasso (L1), Ridge nunca hace que un coeficiente sea exactamente cero. Los hace diminutos, pero mantiene todas las variables en el modelo.



Regularización

Elastic net

$$J(\mathbf{w}) = \text{MSE} + \lambda \left(\alpha \sum_{j=1}^p |w_j| + (1 - \alpha) \sum_{j=1}^p w_j^2 \right)$$

λ controla la **fuerza total de la regularización**.

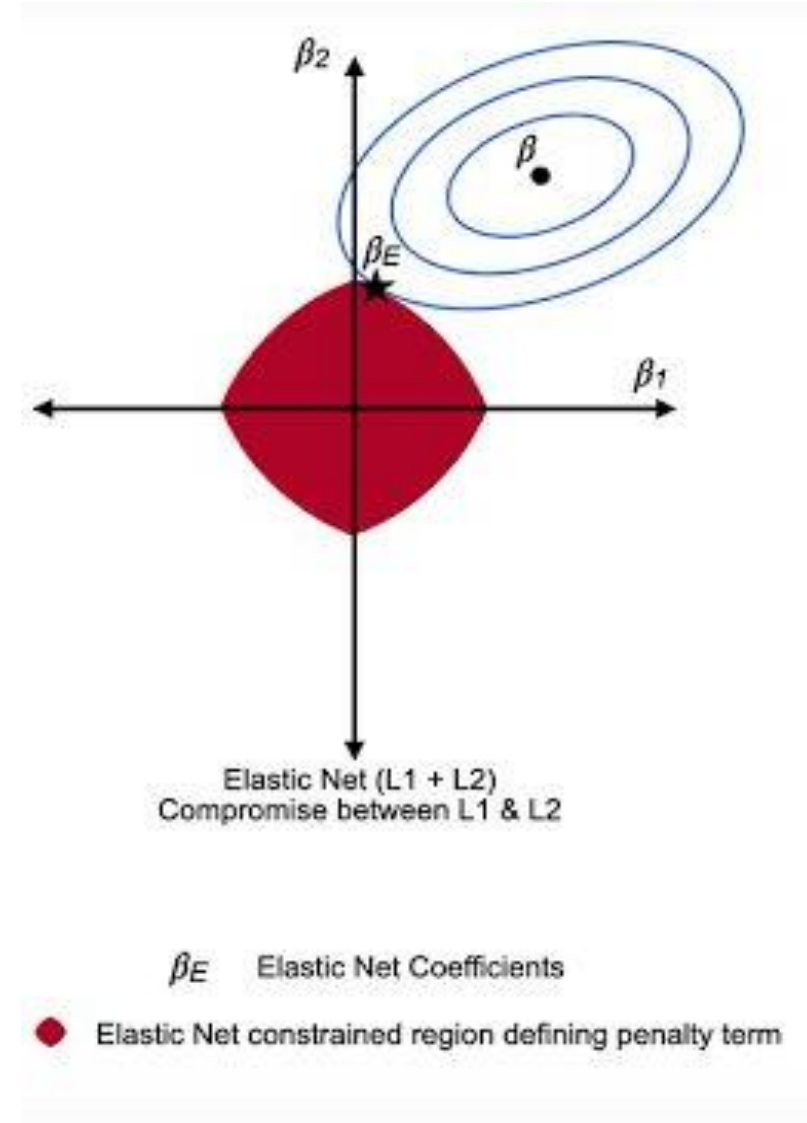
$\alpha \in [0, 1]$ controla la mezcla entre Lasso y Ridge.

Si $\alpha = 1 \rightarrow$ **Lasso**.

Si $\alpha = 0 \rightarrow$ **Ridge**.

Si $0 < \alpha < 1 \rightarrow$ **Elastic Net**.

Algunos pesos pueden quedar exactamente en 0, mientras que los demás tienden a ser más pequeños y estables.



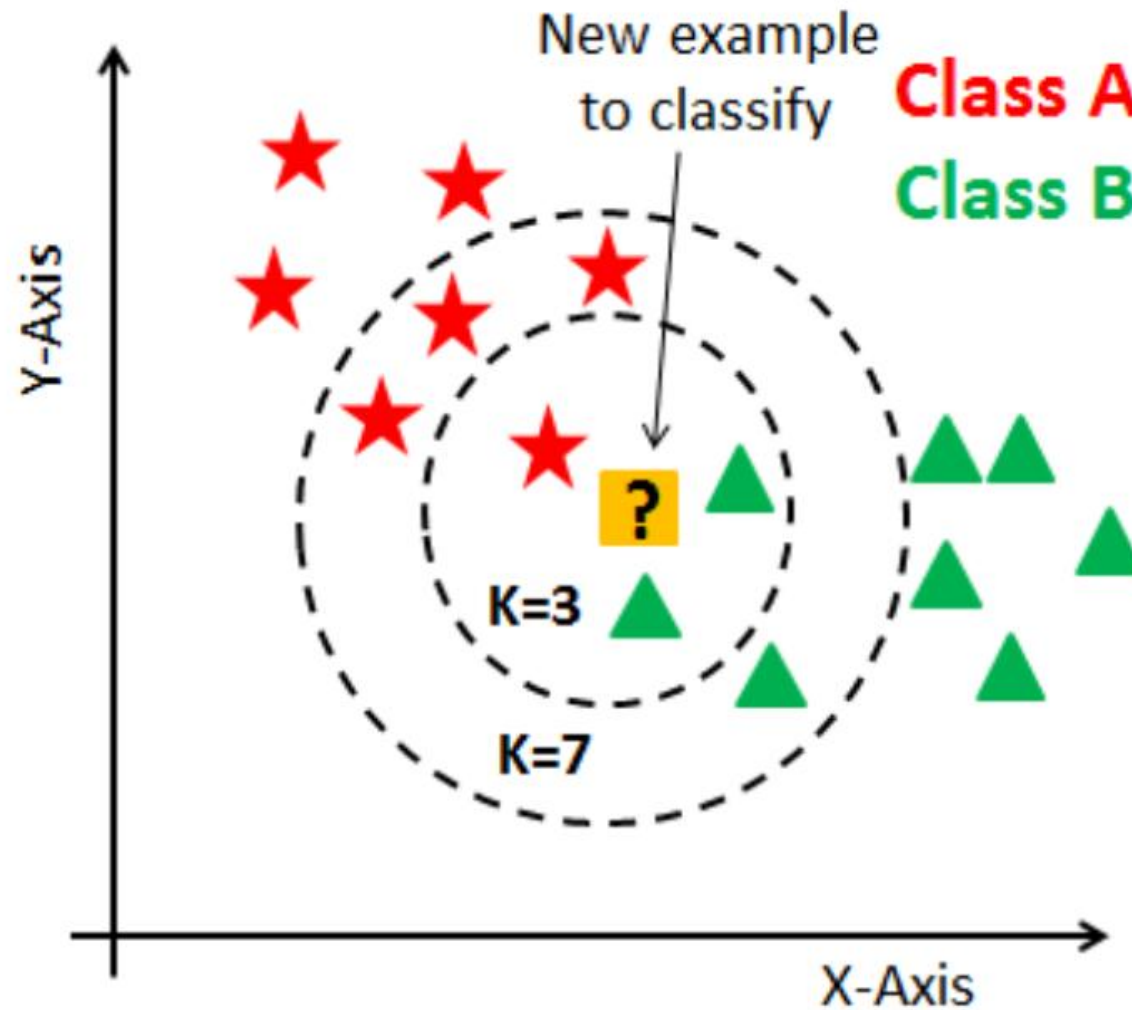
K-Nearest Neighbors (KNN) es un algoritmo de aprendizaje supervisado utilizado principalmente para clasificación y regresión. Su idea fundamental es muy sencilla:

- Para hacer una predicción sobre un nuevo dato, busca los (K) ejemplos más cercanos en el conjunto de entrenamiento y utiliza sus valores para realizar la predicción.

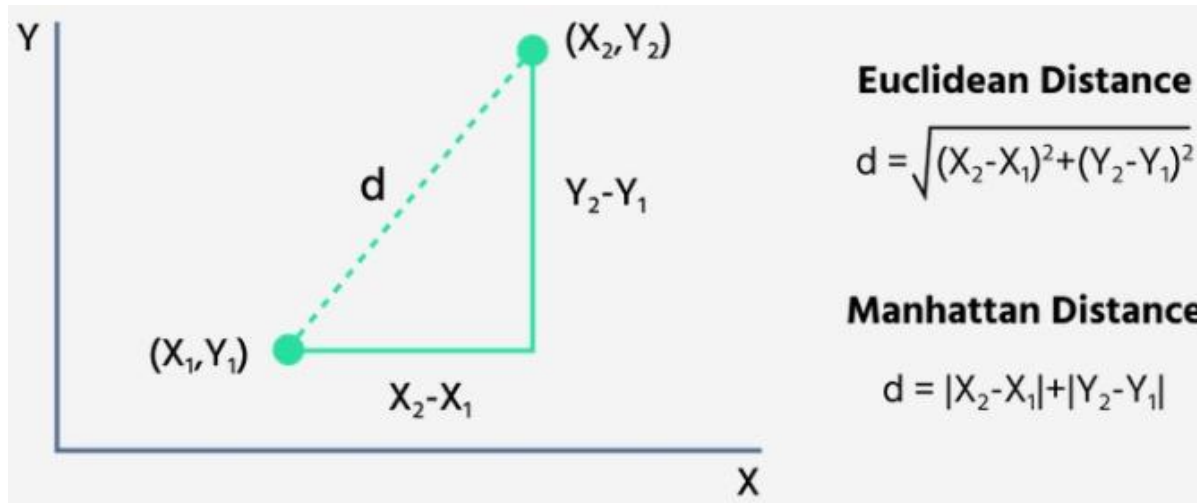
KNN es un Lazy algorithm = No entrena



KNN



Supongamos que tenemos un nuevo punto (x). Primero calculamos su distancia respecto a los datos de entrenamiento. Una de las métricas más utilizadas es la distancia euclídea:



$$d(X, Y) = \left(\sum_{i=1}^n (x_i - y_i)^2 \right)^{1/2}$$

$$d = [(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2]^{0.5}$$

El algoritmo mide la distancia entre el nuevo punto y todos los demás puntos del conjunto de datos.



Nota: Expresión general para la distancia

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

Si $p=1$: Distancia Manhattan

Si $p=2$: Distancia Euclidiana

Característica	Euclidiana (L_2)	Manhattan (L_1)
Trayectoria	Línea recta (diagonal).	Forma de "escalera" o rejilla.
Sensibilidad	Muy sensible a grandes diferencias (por el cuadrado).	Menos sensible a valores extremos.
Cálculo	Más costosa (requiere raíces y potencias).	Muy rápida (solo sumas y restas).
Geometría	El camino más corto posible.	Suma de catetos.

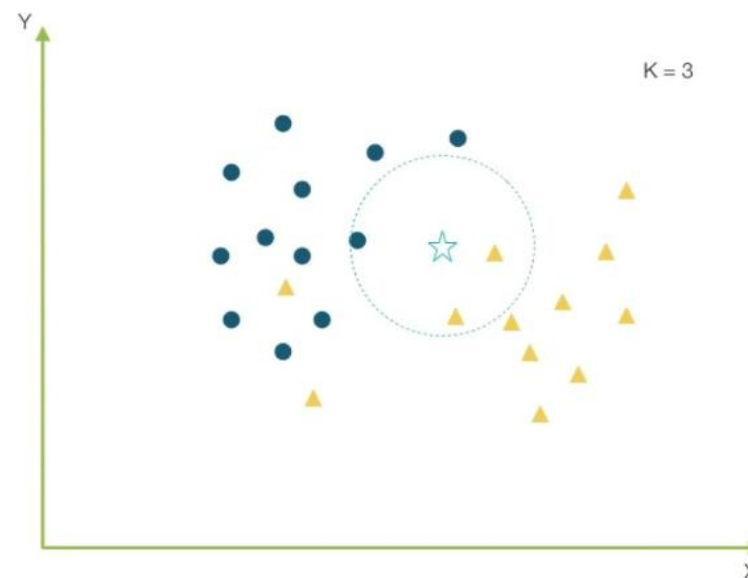


Funcionamiento general:

Identificar los vecinos: Seleccionar los k puntos que tengan la distancia más corta hacia el nuevo dato.

Votación (Clasificación) o Promedio (Regresión)

- En clasificación, el nuevo punto se asigna a la clase más frecuente entre sus k vecinos.
- En regresión, se calcula el promedio de los valores de los k vecinos para dar un resultado.



Importancia de k:

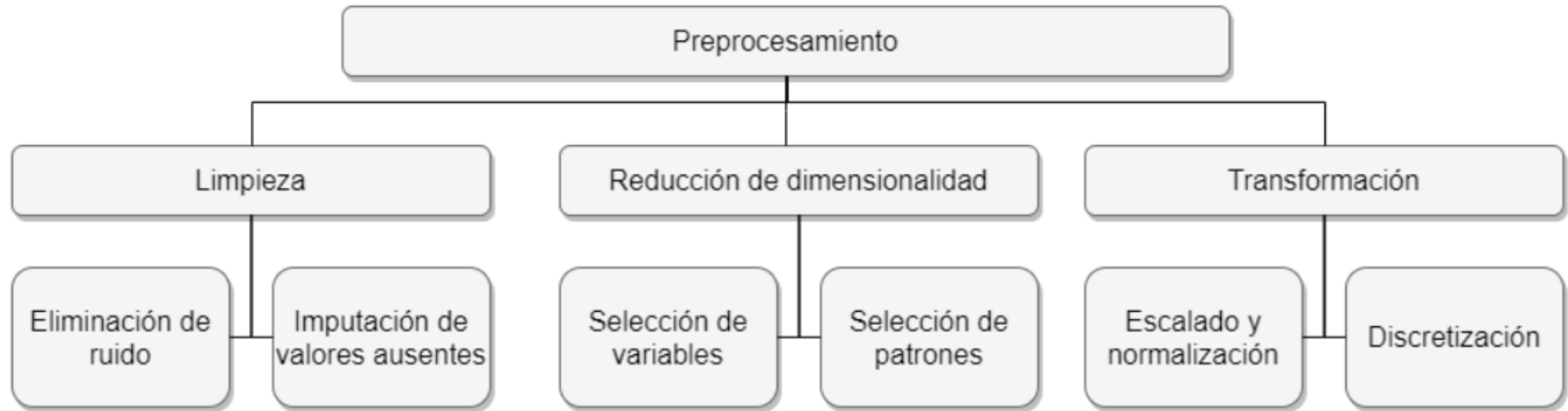
- Si k es muy pequeño (ej. $k=1$), el modelo es muy sensible al ruido y puede generar **overfitting**.
- Si k es muy grande, el modelo se vuelve muy general y puede ignorar patrones locales importantes (**underfitting**).

Notas Importantes:

- Como k-NN se basa en distancias, es fundamental escalar o normalizar las variables.
- A medida que el conjunto de datos crece, k-NN se vuelve más lento. Esto se debe a que, para cada predicción, debe calcular la distancia contra todos los puntos existentes.



Pipelines



Pipelines

Secuencia de transformaciones de datos que, generalmente, finaliza con un estimador, implementa los métodos `.fit()` y `.predict()`. Previene el sesgo a los datos de entrenamiento

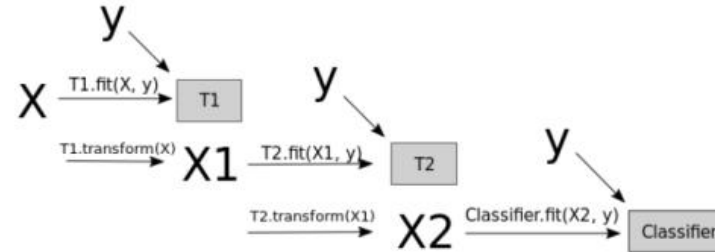
```
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', MinMaxScaler()),
    ('clf', KNeighborsClassifier(n_neighbors = 3))
])
```

```
pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```

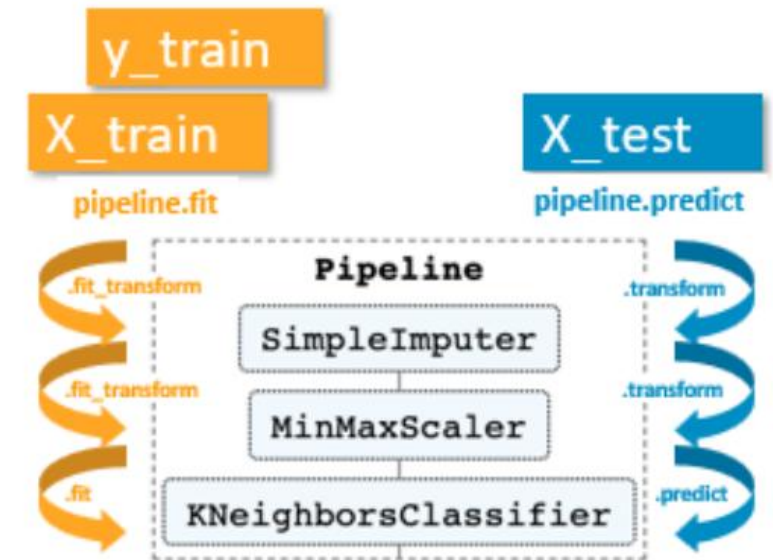
```
pipe = make_pipeline(T1(), T2(), Classifier())
```



```
pipe.fit(X, y)
```



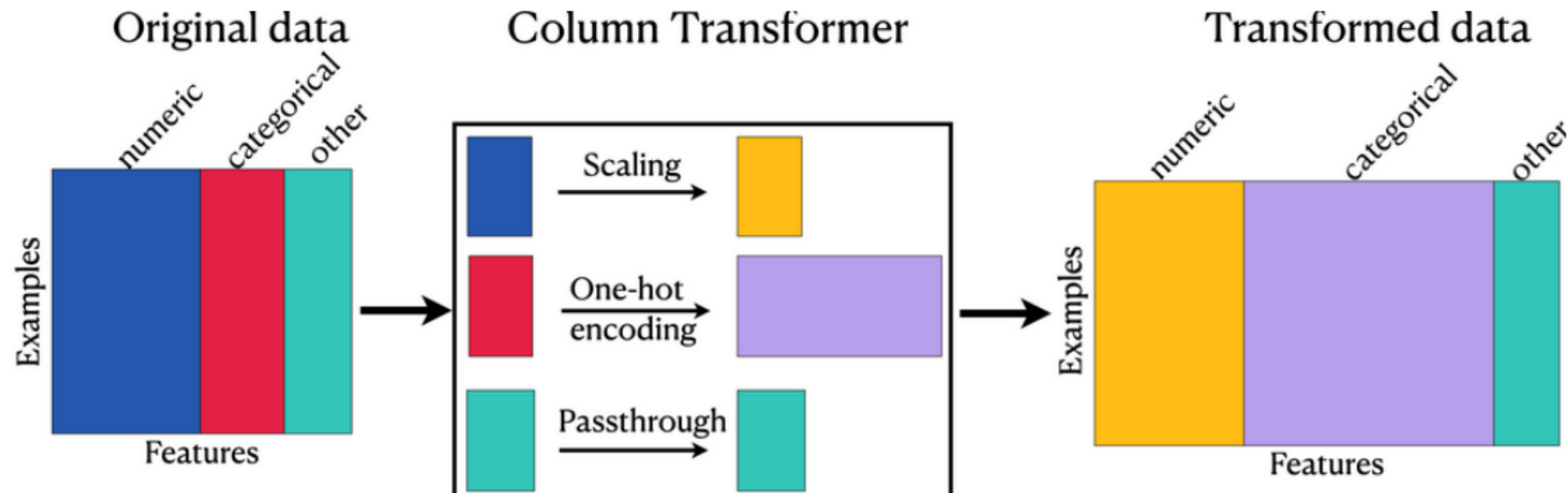
```
pipe.predict(X')
```



Pipelines

`ColumnTransformer` aplica las transformaciones a las columnas de un arreglo de Numpy o a un DataFrame de Pandas. Implementa los métodos `.fit()` y `.transform()`.

- Permite aplicar transformaciones a subconjuntos de características (numérica, categóricas, texto), de forma individual.
- Las características generadas por cada transformación se concatenarán para conformar un solo grupo de características



Pipelines

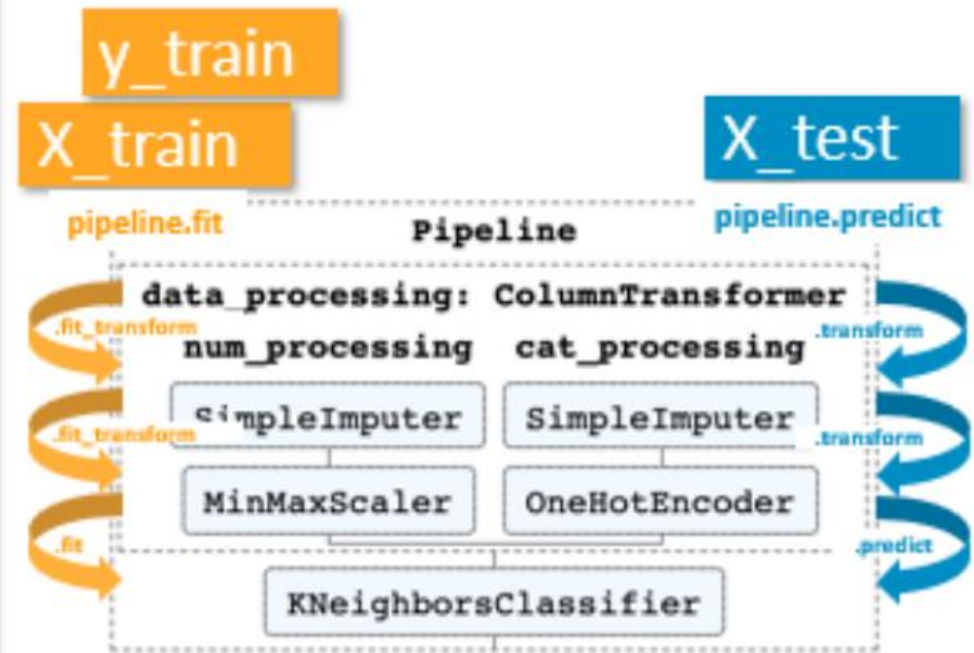
```
numerical_processing = Pipeline([
    ('num_imputer', SimpleImputer(strategy='mean')),
    ('num_scaler', MinMaxScaler())])

categorical_processing = Pipeline([
    ('cat_imputer', Imputer(strategy='constant', fill_value='missing')),
    ('cat_encoder', OneHotEncoder(handle_unknown='ignore'))])

processor = ColumnTransformer(transformers=[
    ('num_processing', numerical_processing, ('feature1', 'feature3')),
    ('cat_processing', categorical_processing, ('feature0', 'feature2'))])

pipeline = Pipeline([
    ('data_processing', processor),
    ('estimator', KNeighborsClassifier())])

pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```



Pipelines

Ejercicios de Clase

